Assignment 5: Data Visualization

Isabelle Kuehn

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy `NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv` version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the `NEON_NIWO_Litter_mass_trap_Processed.csv` version, again from the Processed_KEY folder).

2. Make sure R is reading dates as date format; if not change the format to date.

```
#1
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.0     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/R/RemoteRepositoryClone
```

```
library(lubridate)
library(dplyr)
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

```
library(ggthemes)
```

```
##
## Attaching package: 'ggthemes'
##
## The following object is masked from 'package:cowplot':
##
##     theme_map
```

```
library(ggplot2)
library(viridis)
```

```
## Loading required package: viridisLite
```

```
library(RColorBrewer)
library(colormap)
getwd()
```

```
## [1] "/home/guest/R/RemoteRepositoryClone"
```

```
nutrients.data <- read.csv(file = here(
  "./Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv"),
  stringsAsFactors = TRUE)
litter.data <- read.csv(file=here(
  "./Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv"),
  stringsAsFactors = TRUE)


#2
nutrients.data$sampledate <- as.Date(nutrients.data$sampledate,
                                     format = "%Y-%m-%d")
litter.data$collectDate <- as.Date(litter.data$collectDate,
                                   format = "%Y-%m-%d")
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

```
#3

my_theme <- theme_base()  +
  theme(
    #line =              element_line(color = "darkslategray"),
    #rect =              element_rect(),
    #text =              element_text(),

    # Modified inheritance structure of text element
    plot.title =        element_text(color = "darkblue", size = 14),
    axis.title.x =      element_text(color = "black", size = 12),
    axis.title.y =      element_text(color = "black", size = 12),
    axis.text =         element_text(color = "black"),

    # Modified inheritance structure of line element
    axis.ticks =        element_line(color = "darkblue"),
    panel.grid.major =  element_line(color = "lightgrey"),
    panel.grid.minor =  element_line(color = "lightgrey", linetype = "dotted"),

    # Modified inheritance structure of rect element
    plot.background =   element_rect(fill = "aliceblue"),
    panel.background =  element_rect(fill = "white"),
    legend.key =        element_rect(fill = "azure2"),

    legend.position = 'top',

    )
```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (`tp_ug`) by phosphate (`po4`), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

```
#4

tp_po4_plot <- nutrients.data %>%
  ggplot(aes(x = po4, y = tp_ug, color = lakename)) +
  geom_point() +
```

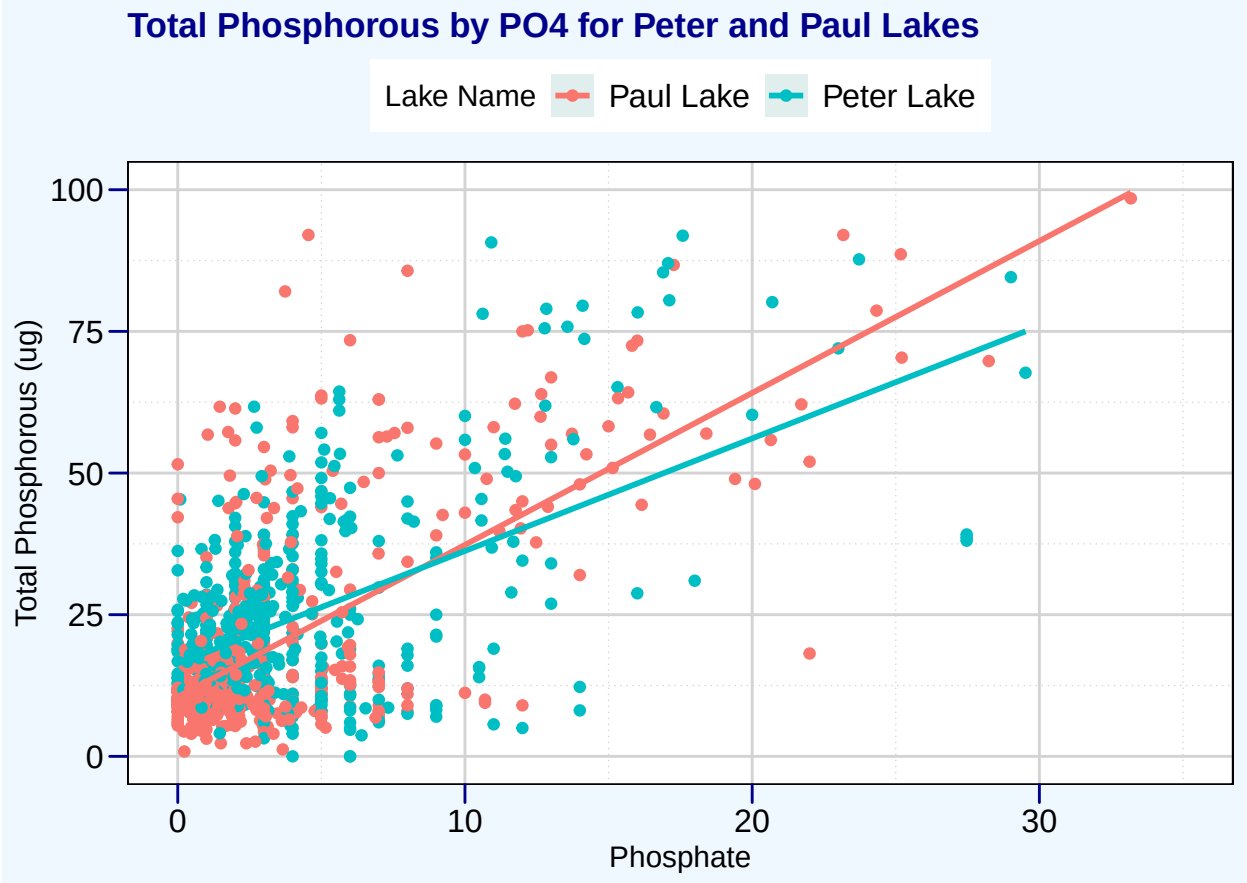
```
  geom_smooth(method = "lm", se = FALSE) +
  xlim(0,35) +
  ylim(0,100) +
  ylab("Total Phosphorous (ug)") +
  xlab("Phosphate") +
  labs(title = "Total Phosphorous by PO4 for Peter and Paul Lakes",
       color = "Lake Name") +
  my_theme +
  theme(legend.title = element_text(size = 12))

tp_po4_plot
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 21967 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 21967 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

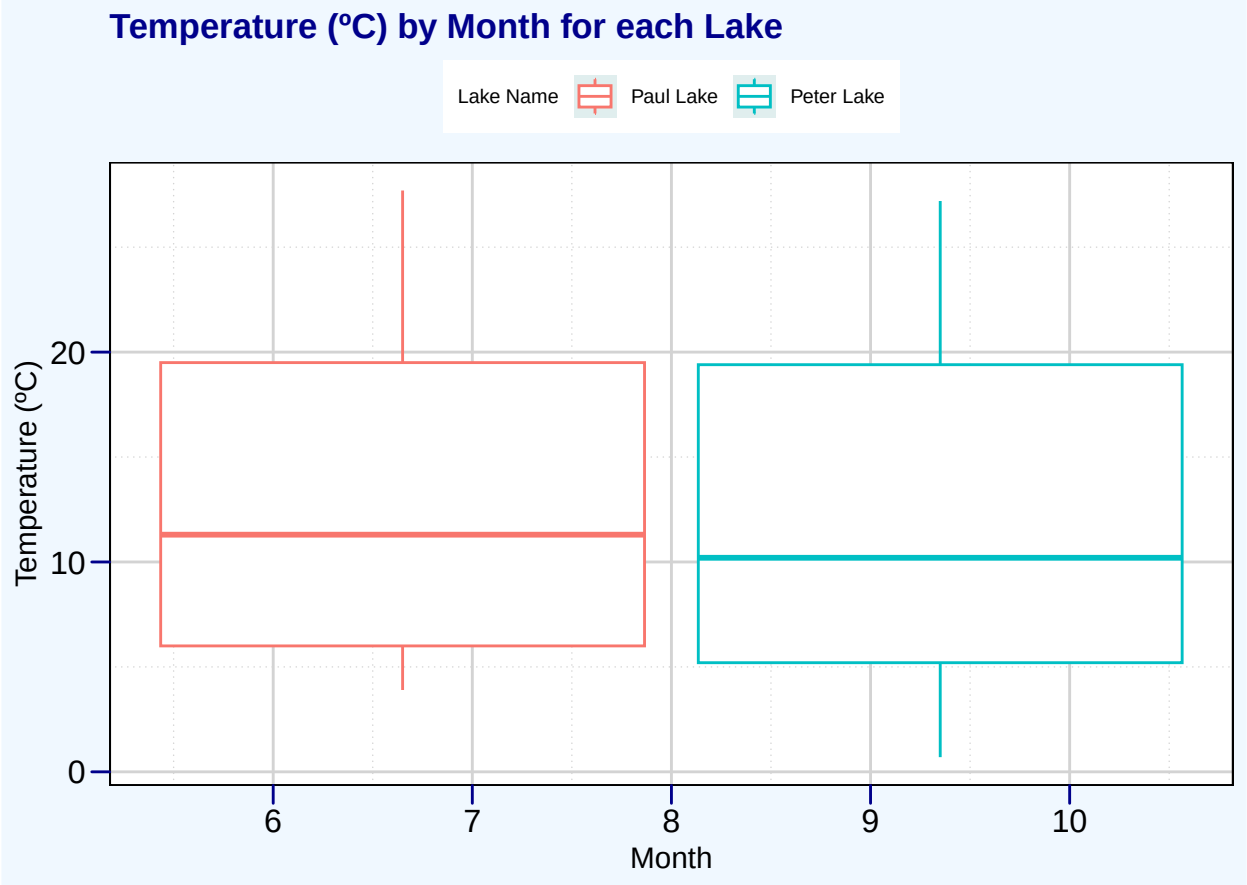


5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

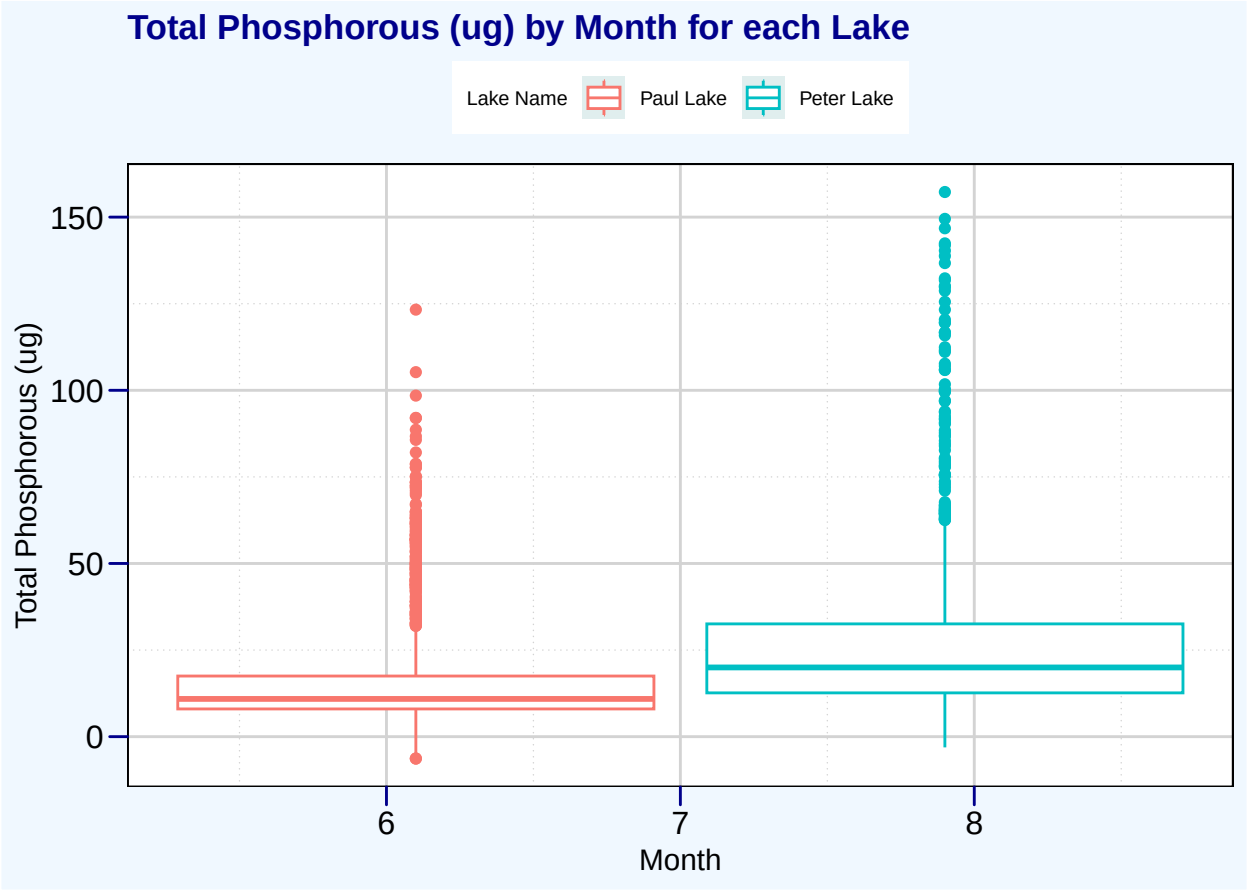
```
#5
temp_boxplot <- nutrients.data %>%
  ggplot(aes(x = month, y = temperature_C, color = lakename)) +
  geom_boxplot() +
  ylab("Temperature (ºC)") +
  xlab("Month") +
  labs(title = "Temperature (ºC) by Month for each Lake", color = "Lake Name") +
  my_theme +
  theme(axis.title = element_text(size = 10),
        legend.position = "top",
        legend.text = element_text(size = 8),
        legend.title = element_text(size = 8))
temp_boxplot
```

```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## (‘stat_boxplot()‘).
```



```
tp_boxplot <- nutrients.data %>%
  ggplot(aes(x = month, y = tp_ug, color = lakename)) +
  geom_boxplot() +
  ylab("Total Phosphorous (ug)") +
  xlab("Month") +
  labs(title = "Total Phosphorous (ug) by Month for each Lake",
       color = "Lake Name") +
  my_theme +
  theme(axis.title = element_text(size = 10),
        legend.position = "top",
        legend.text = element_text(size = 8),
        legend.title = element_text(size = 8))
tp_boxplot
```

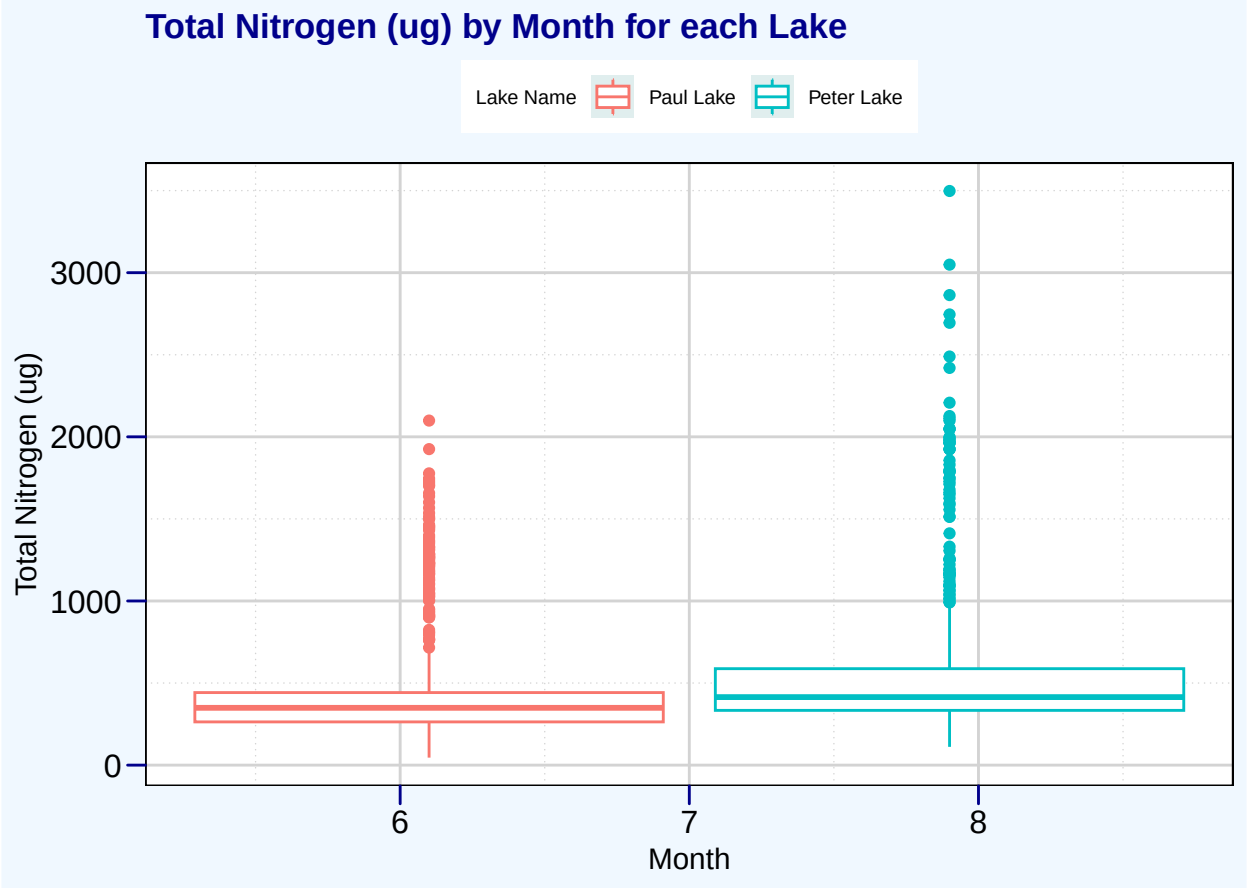
```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```




```
tn_boxplot <- nutrients.data %>%
  ggplot(aes(x = month, y = tn_ug, color = lakename)) +
  geom_boxplot() +
  ylab("Total Nitrogen (ug)") +
  xlab("Month") +
  labs(title = "Total Nitrogen (ug) by Month for each Lake",
```

```
       color = "Lake Name") +
  my_theme +
  theme(axis.title = element_text(size = 10),
        legend.position = "top",
        legend.text = element_text(size = 8),
        legend.title = element_text(size = 8))
tn_boxplot
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



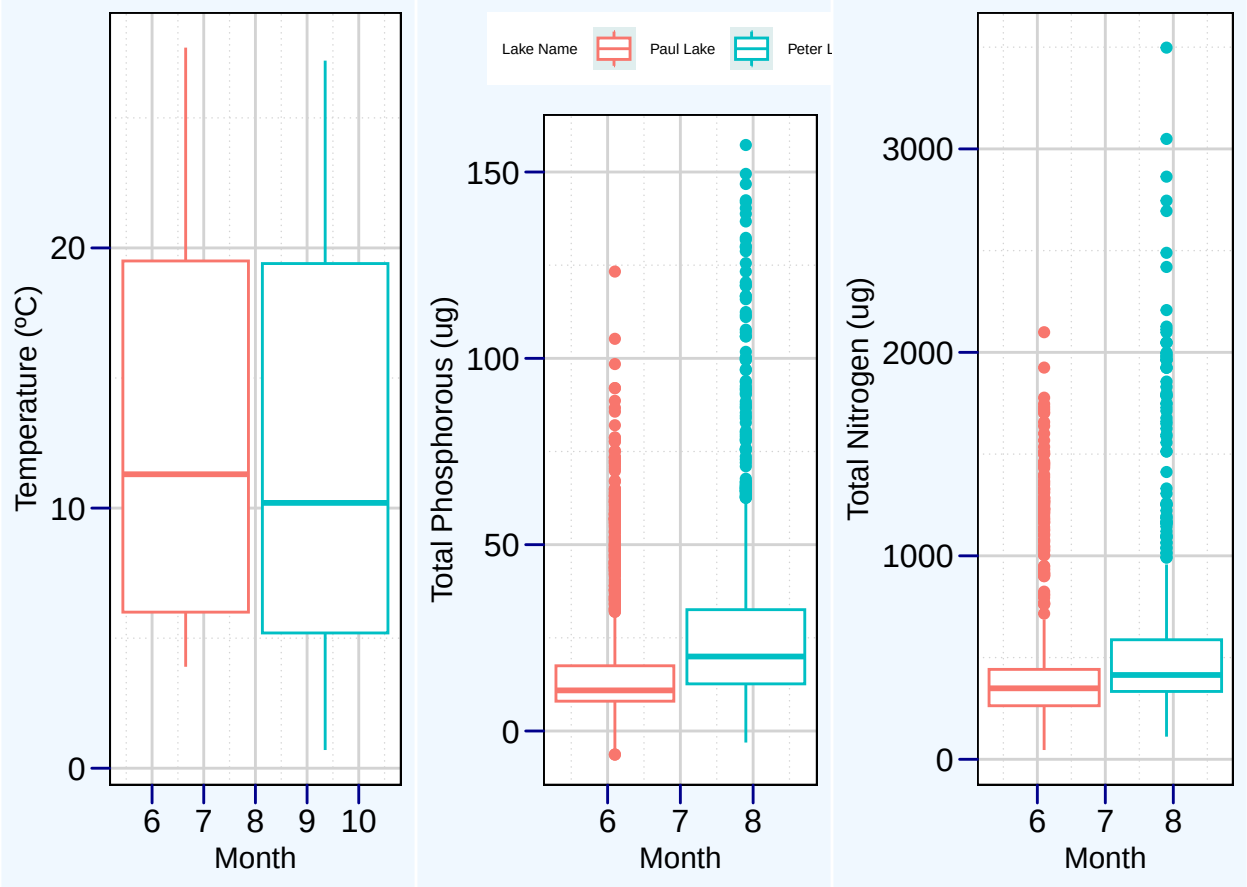

```
temp_boxplot_edit <- temp_boxplot + theme(plot.title = element_blank(),
        legend.position = "none",
        legend.text = element_blank(),
        legend.title = element_blank())
tp_boxplot_edit <- tp_boxplot + theme(plot.title = element_blank(),
        legend.position = "top",
        legend.text = element_text(size = 6),
        legend.title = element_text(size = 6))
tn_boxplot_edit <- tn_boxplot + theme(plot.title = element_blank(),
        legend.position = "none",
        legend.text = element_blank(),
        legend.title = element_blank())
```

```
plot_grid(temp_boxplot_edit, tp_boxplot_edit, tn_boxplot_edit,
          ncol = 3
          )
```

```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```




```
# I feel like this plot could've been tidier, but I'm not really sure how else
# to organize it with cowplot. I looked at the alignment website but couldn't
# find much else.
```

Question: What do you observe about the variables of interest over seasons and between lakes?

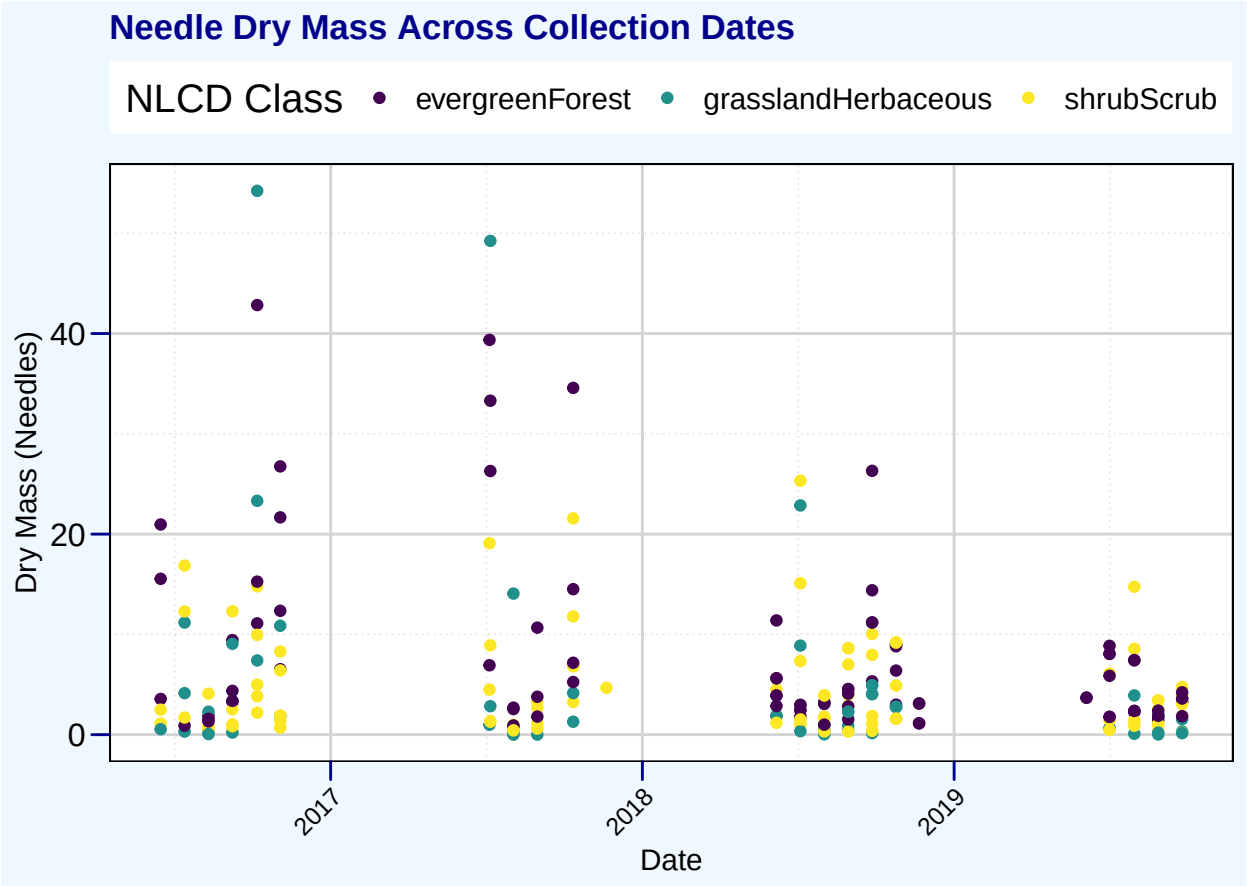
> Answer: Over seasons both total phosphorous and total nitrogen levels increased slightly, while temperature decreased as expected. Looking at the differences between lakes, Paul Lake measurements seem to have been recorded in earlier months than Peter lake data.

6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the "Needles" functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)

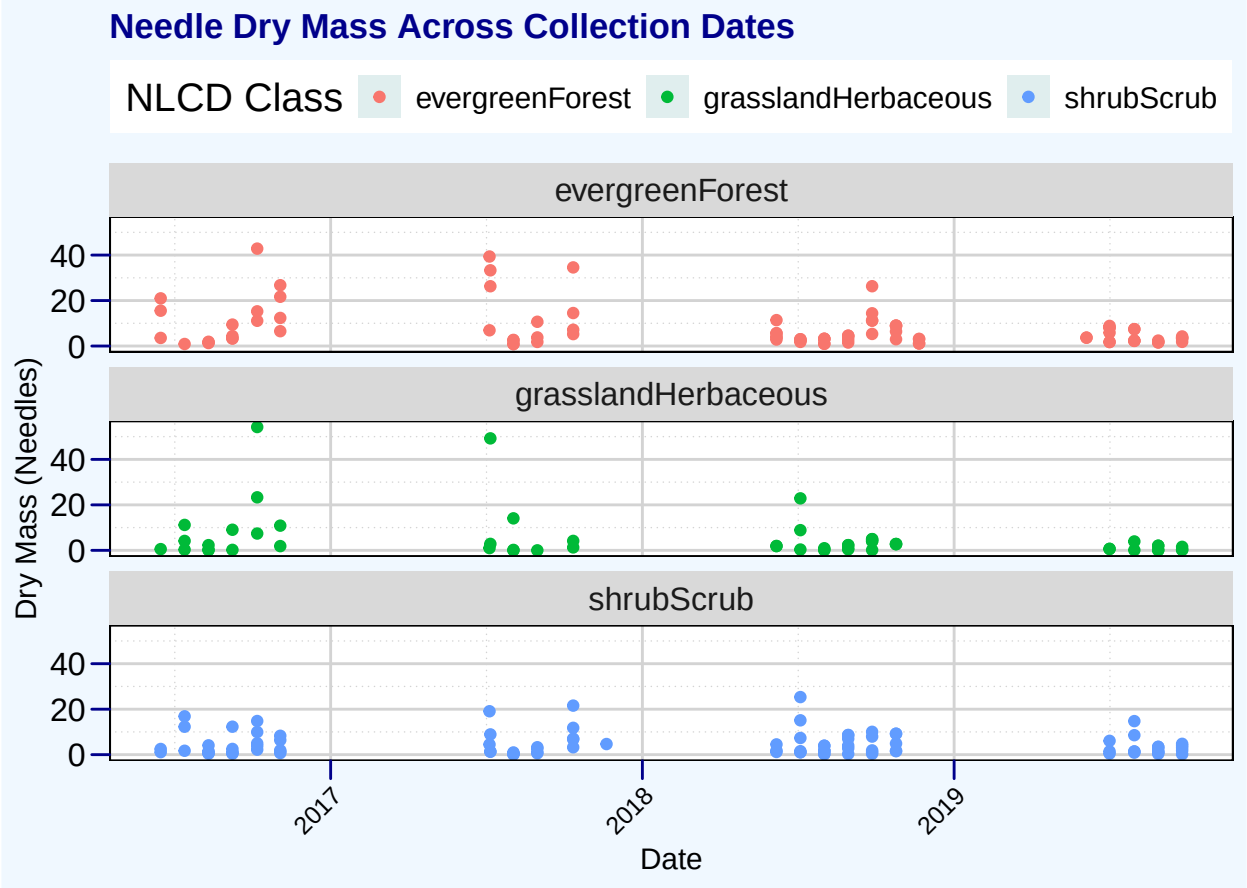
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

```r
#6
needle.data <- litter.data %>%
  filter(litter.data$functionalGroup == "Needles")

Litter.plot <- needle.data %>%
  ggplot(aes(x = collectDate, y = dryMass, color = nlcdClass))+
  geom_point()+
  ylab("Dry Mass (Needles)") +
  xlab("Date") +
  labs(title = "Needle Dry Mass Across Collection Dates",
       color = "NLCD Class") +
  scale_color_viridis_d(option = "viridis") +
  my_theme +
  theme(legend.text = element_text(size = 12), legend.key = element_blank(),
        legend.position = 'top',
        axis.text.x = element_text(angle = 45, hjust = 1, size = 10))
print(Litter.plot)
```



```r
#7
Litter.facet.plot <- needle.data %>%
  ggplot(aes(x = collectDate, y = dryMass, color = nlcdClass))+
  geom_point()+
  facet_wrap(facets= vars(nlcdClass),
             nrow=3) +
  ylab("Dry Mass (Needles)") +
  xlab("Date") +
  labs(title = "Needle Dry Mass Across Collection Dates",
       color = "NLCD Class") +
  my_theme +
  theme(legend.text = element_text(size = 12), legend.position = 'top',
        axis.text.x = element_text(
    angle = 45,  hjust = 1, size = 10))
print(Litter.facet.plot)
```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: I believe the plot created in question 6 is more effective becasue the facet wrap separates each dataset making comparison along the changing y-axis harder to identify. For each NLCD Class, the dry mass needle data has the same date of collection which means each dataset will have data present at the same points along the x-axis. As a result, the most important variable to measure between datasets is the dry mass of needles, as that is the variable changing between each NLCD Class. Looking at the plot in question 7, facet wrapping makes comparison between classes more difficult making it less effective than the graph produced in question 6. The plot for

question 6 shows direct similarities and differences between dry mass collection because all data is on the same plot. While there is overlap, it isn't enough to detract from the results, while the separation from the facet wrapping is. I find the facet wrapped plot to be the best representation for individual analysis of each class, but for the effectiveness of comparing classes, question 6 has the most effective visual representation.